Modelos de Concorrência num Microserviço Java: Resultados Preliminares

Pedro Gomes
Programação Paralela e Concorrente
Mestrado em Engenharia Informática
Faculdade de Ciências, Universidade de Lisboa
Lisboa, Portugal
fc67991@alunos.ciencias.ulisboa.pt

Abstract

Este relatório apresenta resultados preliminares da comparação entre platform threads, virtual threads e I/O assíncrono num agregador Spring Boot que chama um serviço HTTP simulado (RQ1 da proposta). Num piloto com k6 a 10, 100 e 500 utilizadores virtuais (VUs), os três modelos tiveram desempenho próximo até 100 VUs. A 500 VUs, virtual e async atingiram cerca de 404 pedidos/s com p99 de 1,13 s, contra 184 pedidos/s e p99 de 2,84 s nas platform threads, um padrão consistente com a hipótese H1. Com uma medição por condição e na mesma máquina, os resultados são exploratórios e não permitem conclusões gerais.

1 Contexto e questões de investigação

Os microserviços passam grande parte do tempo à espera de respostas externas, pelo que o modelo de concorrência condiciona o seu desempenho sob carga. A proposta define duas questões e hipóteses:

RQ1. Como variam o throughput e a latência dos três modelos com o aumento dos pedidos concorrentes? **H1.** Sob concorrência elevada e carga I/O-bound, virtual threads e I/O assíncrono superam platform threads em throughput e latência de cauda.

RQ2. Como influenciam a latência externa, chamadas por pedido e trabalho de CPU o desempenho relativo dos modelos? **H2.** As vantagens de virtual threads e async diminuem com o aumento de trabalho de CPU. Este relatório cobre o piloto da RQ1; a RQ2 é o trabalho seguinte.

2 Arquitetura e implementação

O protótipo tem um agregador Spring Boot (porta 8081) e um stub (porta 8080) que aguarda 1 000 ms e devolve JSON. O stub usa `spring.threads.virtual.enabled=true` para evitar que o serviço simulado limite a carga.

No módulo agregador, implementaram-se os três modelos de concorrência para o mesmo percurso: (i) com **platform threads**, a thread do Tomcat bloqueia na chamada externa; (ii) com **virtual threads**, o pedido corre numa virtual thread dedicada via `CompletableFuture`; (iii) no modelo **assíncrono**, `HttpClient.sendAsync()` liberta a thread durante o I/O. O percurso e trabalho funcional são idênticos nos três modos [4]. O código-fonte está disponível em [2].

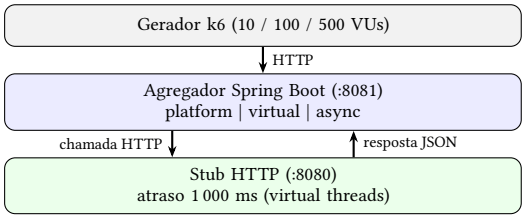


Figure 1. Arquitetura do ensaio. Os três endpoints do agregador fazem uma chamada equivalente ao stub.

3 Metodologia do piloto

O k6 executou uma vez cada combinação de modo e concorrência (10, 100 e 500 VUs): subida/aquecimento de 10 s, carga constante de 30 s, descida de 5 s e timeout de 15 s. Foram recolhidos throughput, p50/p95/p99 e erros, usando as métricas incorporadas do k6 [3]. O ambiente de ensaio foi Windows 10 Home (64-bit), CPU Intel Core i5-10400F (6 núcleos / 12 threads @ 2,90 GHz), 16 GB de RAM DDR4, Eclipse Adoptium Temurin OpenJDK 21.0.13 (LTS) e Maven 3.10.0. Gerador, agregador e stub correram na mesma máquina.

O piloto é um subconjunto deliberado da proposta (Tabela 1): serve para validar a metodologia e reduzir o âmbito antes das medições finais.

Table 1. Proposta versus piloto.

Aspeto	Proposta	Piloto
Concorrência	10, 100, 1000, 2000	10, 100, 500 VUs
Latência stub	fixa, variável, cauda	fixa (1 000 ms)
Chamadas	1, 3, 5 por pedido	1
Carga de CPU	nenhum, moderado, alto	nenhum
Métricas	throughput, latência, RAM	throughput, latência
Gerador	máquina externa	mesma máquina
Repetições	várias com dispersão	uma única execução

4 Resultados

Até 100 VUs, as três variantes apresentam throughput e latências próximos (Figuras 2 e 3). A 500 VUs (Tabela 2), as platform threads ficam perto de 184 pedidos/s e p99 de 2,84 s; virtual e async ficam perto de 404 pedidos/s e p99 de 1,13 s, ou seja, cerca de 2,2× o throughput e 2,5× menos latência de cauda. Os nove resultados guardados registam 0% de erros. Entre virtual e async, as diferenças são inferiores a 3% em

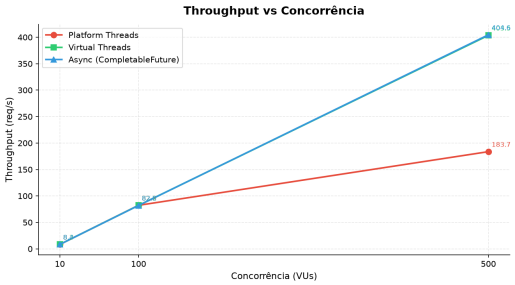


Figure 2. Throughput por concorrência.

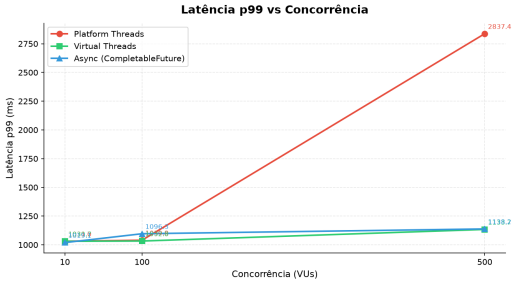


Figure 3. Latência p99 por concorrência.

Table 2. Resultados a 500 VUs (uma execução por modo).

Modo	req/s	p50 (ms)	p95 (ms)	p99 (ms)
Platform	183,71	2 477,89	2 793,12	2 837,45
Virtual	403,53	1 018,53	1 091,43	1 133,70
Async	404,58	1 015,83	1 060,21	1 138,17

todas as métricas, mas, sem repetições, não é possível saber se são ruído; a proximidade não demonstra equivalência nem superioridade geral.

Relação com estudos anteriores. Dundov [1] observou que virtual threads em Spring Boot superam platform threads sob carga I/O-bound elevada; Pandita [5] reportou ganhos equivalentes em workloads de alto débito com I/O bloqueante. Os resultados deste piloto alinham-se com esses estudos: a vantagem de ≈2,2× em throughput e ≈2,5× em latência de cauda surge a 500 VUs, quando o pool de platform threads se torna o gargalo. A convergência dos três modelos até 100 VUs é também consistente com a literatura: a diferença de desempenho só se manifesta quando a concorrência pressiona os recursos de threading.

Relação com H1. Os resultados a 500 VUs são consistentes com H1: o modelo bloqueante em platform threads satura e os outros dois não. Mas H1 refere concorrência elevada e a proposta prevê 1000 e 2000 VUs, ainda não testados, e com uma repetição por condição não há estimativa de variabilidade nem teste estatístico. H1 fica, por agora, apenas indiciada, não confirmada.

5 Ameaças à validade

- **Mesma máquina.** O gerador competiu por CPU e memória com os serviços no Intel Core i5-10400F (16 GB de RAM); para as medições finais o gerador deverá correr noutra máquina como previsto na proposta.
- **Uma repetição, ordem fixa.** Sem repetições não há dispersão nem estimativa de variabilidade, e a ordem fixa pode introduzir efeitos de aquecimento da JVM.
- **Âmbito do cenário.** O atraso de 1 s é fixo, com uma chamada e sem carga de CPU; o consumo de memória (variável da proposta) ainda não foi medido.
- **Dimensão do pool.** O piloto usou o pool padrão do Tomcat (200 threads); esta dimensão deve ser registada e fixada formalmente para assegurar a reprodutibilidade.

6 Próximos passos

Até à entrega final (2 de dezembro): (i) repetir cada condição várias vezes, alternando ou aleatorizando a ordem dos modos, e registar CPU e memória do sistema; (ii) acrescentar 1000 e 2000 VUs e confirmar que o gerador não é o gargalo, idealmente com o gerador noutra máquina; (iii) fixar formalmente a dimensão do thread pool do Tomcat; (iv) para RQ2, variar um fator de cada vez: atraso e cauda longa, 1/3/5 chamadas ao stub e trabalho de CPU, testando H2; (v) medir o consumo de memória de platform e virtual threads; (vi) reportar a dispersão e usar um teste estatístico adequado, justificando-o.

7 Use of AI tools.

Claude (Anthropic): brainstorming do tema, discussão do desenho experimental, pesquisa de literatura e revisão do relatório.

References

[1] Anja Dundov. 2026. Evaluating Java Virtual Threads for Concurrent Spring Boot and Helidon Services. In *2026 49th MIPRO ICT and Electronics Convention (MIPRO)*. IEEE, Opatija, Croatia. doi:10.1109/MIPRO70003.2026.11591610

[2] Pedro Gomes. 2026. Repositório do projeto PPC: Modelos de Concorrência num Microserviço Java. https://github.com/pedroogomes05/ppc. Acedido: novembro 2026.

[3] Grafana Labs. 2026. k6 Built-in metrics. https://grafana.com/docs/k6/latest/using-k6/metrics/reference/. Acedido: novembro 2026.

[4] OpenJDK. 2023. JEP 444: Virtual Threads. https://openjdk.org/jeps/444. Acedido: novembro 2026.

[5] V. Pandita. 2024. *Benchmarking the Performance of Java Virtual Threads in High-Throughput Workloads.* MSc thesis. National College of Ireland. https://norma.ncirl.ie/8134/